

Number of Restricted paths from first to last node

Google | LC 1786

© Authored by Arpit Raj, LNMIIT Jaipur, July 31, 2026

Problem Understanding

need to go from `1` to `n` and count no. of restricted paths.

a path is restricted if: $\text{dist}(u, n) > \text{dist}(v, n)$

(for edge $u \rightarrow v$)

where $\text{dist}(x, n)$ is shortest distance from node x to node n .

hence only allowed to move to nodes with shorter distance.

hence need to know which neighbour is closer to destination.

Dijkstra Approach Selection

Opt 1

run Dijkstra from every node to n

Opt 2

run Dijkstra from node n to every node

✓ **Opt 2 is faster (and possible since undirected)**

Algorithm Steps

(S1) Build Graph

(S2) run one Dijkstras from n

(S3) every node knows its shortest distance to n

(S4) DFS + memoization using condition `if(dist[curr] > dist[next])`

Implementation

```

class Solution {
public:
    const int MOD=1e9+7;

    int dfs(int curr, vector<vector<pair<int,int>>>& adj, vector<long
long>& dist, vector<int>& dp, int n){

        if(curr==n) return 1;
        if(dp[curr]!=-1) return dp[curr];

        long long ways=0;
        for(auto &it:adj[curr]){
            //sirf unhi node pe jaa skte jaha se manzil paas pade
            if(dist[it.first]<dist[curr]){
                ways=(ways+dfs(it.first,adj,dist,dp,n))%MOD;
            }
        }

        dp[curr]=ways;
        return dp[curr];
    }

    int countRestrictedPaths(int n, vector<vector<int>>>& edges) {
        //basic aadya ye he ki
        //graaph bnao
        //n se djikstra krke har node ki distance nikaal lenge n se
(shortest)
        //dfs+memoization starting from node 1

        vector<vector<pair<int,int>>> adj(n+1);
        vector<long long> dist(n+1,LLONG_MAX);

        for(auto &it:edges){
            int u=it[0];
            int v=it[1];
            int w=it[2];

            adj[u].push_back({v,w});
            adj[v].push_back({u,w});
        }
    }

```

```

        //pq me {cost,node}
        priority_queue<pair<long long,int>,vector<pair<long
long,int>>,greater<pair<long long,int>>> pq;
        pq.push({0,n});
        dist[n]=0;

        while(!pq.empty()){
            int curr=pq.top().second;
            long long currcost=pq.top().first;

            pq.pop();
            if(currcost>dist[curr]){
                continue;
            }

            for(auto &it:adj[curr]){
                int next=it.first;
                long long cost=it.second;

                long long newcost=currcost+cost;

                if(newcost<dist[next]){
                    dist[next]=newcost;
                    pq.push({newcost,next});
                }
            }
        }

        vector<int> dp(n,-1);
        return dfs(1,adj,dist,dp,n);
    }
};

```